

RedFlag

The Fraud Files — Complete SQL Solution

Unlax Academy Minor Project · PayFast Fraud Investigation

This document contains the complete MySQL solution for all 12 fraud patterns in the supplied RedFlag dataset. The queries are structured for MySQL 8.0 and assume the database and table have already been loaded. The findings counts below were validated against the supplied transaction data.

Solution Summary

Pattern	Detection	Validated results	Expected in brief
P1	VELOCITY FRAUD	50	~45–55
P2	ROUND-AMOUNT CLUSTERING	25	25
P3	CARD TESTING	20	20
P4	FAILED-THEN-SUCCEEDED	25	25
P5	ODD-HOUR CONCENTRATION	20	20
P6	MULE ACCOUNTS	30	30
P7	REFUND ABUSE	24	24–25
P8	MERCHANT COLLUSION	15	15
P9	JUST-UNDER-THRESHOLD (STRUCTURING)	20	20
P10	DORMANT-THEN-ACTIVE	26	25–27
P11	VELOCITY SPIKE	42	35–45
P12	GEOGRAPHIC IMPOSSIBILITY	15	15

Dataset validation: 200,594 transactions, 14,755 distinct users, with timestamps from 2024-01-01 through 2024-06-28 in the supplied file. This is consistent with the brief's “approximately 200,000” and “approximately 14,700” checks, although the supplied data does not extend through June 30.

Pattern 1 · VELOCITY FRAUD

Users with 30+ transactions in a single calendar day.

Validated findings: 50 suspect result(s).

Top results: 14556 | 2024-05-28 | 60; 14569 | 2024-04-03 | 60; 14559 | 2024-06-04 | 59

MySQL 8.0 query:

```
SELECT user_id,  
       DATE(txn_time) AS attack_date,  
       COUNT(*) AS daily_txn_count  
FROM transactions  
GROUP BY user_id, DATE(txn_time)  
HAVING COUNT(*) >= 30  
ORDER BY daily_txn_count DESC;
```

Pattern 2 · ROUND-AMOUNT CLUSTERING

Users with 15+ transactions at exactly round-number amounts.

Validated findings: 25 suspect result(s).

Top results: 14535 | 30; 14534 | 30; 14533 | 30

MySQL 8.0 query:

```
SELECT user_id,  
       COUNT(*) AS round_txn_count  
FROM transactions  
WHERE amount IN (100.00, 200.00, 500.00, 1000.00,  
                 2000.00, 5000.00, 10000.00)  
GROUP BY user_id  
HAVING COUNT(*) >= 15  
ORDER BY round_txn_count DESC;
```

Pattern 3 · CARD TESTING

Users with 30+ transactions below ₹10 on one day.

Validated findings: 20 suspect result(s).

Top results: 14556 | 2024-05-28 | 60; 14569 | 2024-04-03 | 60; 14559 | 2024-06-04 | 59

MySQL 8.0 query:

```
SELECT user_id,
       DATE(txn_time) AS attack_date,
       COUNT(*) AS tiny_txn_count
FROM transactions
WHERE amount < 10.00
GROUP BY user_id, DATE(txn_time)
HAVING COUNT(*) >= 30
ORDER BY tiny_txn_count DESC;
```

Pattern 4 · FAILED-THEN-SUCCEEDED

Users with 20+ failed transactions followed within two minutes by a successful transaction of the same amount.

Validated findings: 25 suspect result(s).

Top results: 14595 | 35; 14593 | 34; 14576 | 33

MySQL 8.0 query:

```
SELECT f.user_id,
       COUNT(*) AS failed_then_success_pairs
FROM transactions AS f
JOIN transactions AS s
  ON s.user_id = f.user_id
 AND s.status = 'SUCCESS'
 AND s.amount = f.amount
 AND s.txn_time > f.txn_time
 AND TIMEDIFF(MINUTE, f.txn_time, s.txn_time) <= 2
WHERE f.status = 'FAILED'
GROUP BY f.user_id
HAVING COUNT(*) >= 20
ORDER BY failed_then_success_pairs DESC;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Pattern 5 · ODD-HOUR CONCENTRATION

Users with at least 30 transactions and at least 80% of activity between 02:00 and 04:59.

Validated findings: 20 suspect result(s).

Top results: 14606 | 52 | 49 | 0.9423076923076923; 14609 | 48 | 45 | 0.9375; 14608 | 63 | 58 | 0.9206349206349206

MySQL 8.0 query:

```
SELECT user_id,
       COUNT(*) AS total_txns,
       SUM(CASE WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1 ELSE 0 END) AS odd_hour_txns,
       ROUND(
         SUM(CASE WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1 ELSE 0 END)
         / COUNT(*), 4
       ) AS odd_hour_ratio
FROM transactions
GROUP BY user_id
HAVING COUNT(*) >= 30
      AND SUM(CASE WHEN HOUR(txn_time) BETWEEN 2 AND 4 THEN 1 ELSE 0 END) / COUNT(*) >= 0.80
ORDER BY odd_hour_ratio DESC;
```

Pattern 6 · MULE ACCOUNTS

Users with at least five CREDIT→DEBIT instances within 30 minutes, with the debit at least 70% of the credit amount.

Validated findings: 30 suspect result(s).

Top results: 14645 | 15; 14643 | 15; 14640 | 15

MySQL 8.0 query:

```
SELECT c.user_id,
       COUNT(*) AS qualifying_pairs
FROM transactions AS c
WHERE c.txn_type = 'CREDIT'
      AND c.payment_mode = 'NETBANKING'
      AND EXISTS (
        SELECT 1
        FROM transactions AS d
        WHERE d.user_id = c.user_id
              AND d.txn_type = 'DEBIT'
              AND d.payment_mode = 'UPI'
              AND d.txn_time > c.txn_time
              AND TIMEDIFF(MINUTE, c.txn_time, d.txn_time) <= 30
              AND d.amount >= 0.70 * c.amount
      )
GROUP BY c.user_id
HAVING COUNT(*) >= 5
ORDER BY qualifying_pairs DESC;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Pattern 7 · REFUND ABUSE

Users with 20+ transactions and a refund ratio above 40%.

Validated findings: 24 suspect result(s).

Top results: 14662 | 39 | 25 | 0.6410256410256411; 14670 | 50 | 32 | 0.64; 14665 | 36 | 23 | 0.6388888888888888

MySQL 8.0 query:

```
SELECT user_id,
       COUNT(*) AS total_txns,
       SUM(CASE WHEN txn_type = 'REFUND' THEN 1 ELSE 0 END) AS refund_count,
       ROUND(
         SUM(CASE WHEN txn_type = 'REFUND' THEN 1 ELSE 0 END) / COUNT(*), 4
       ) AS refund_ratio
FROM transactions
GROUP BY user_id
HAVING COUNT(*) >= 20
      AND SUM(CASE WHEN txn_type = 'REFUND' THEN 1 ELSE 0 END) / COUNT(*) > 0.40
ORDER BY refund_ratio DESC;
```

Pattern 8 · MERCHANT COLLUSION

Merchants where the top five users account for more than 60% of total transaction value.

Validated findings: 15 suspect result(s).

Top results: 12 | 2175353.42 | 2177212.35 | 0.9991461880142283; 8 | 1611603.39 | 1613683.91 | 0.9987107016516016; 13 | 1915648.42 | 1918510.08 | 0.9985083945975409

MySQL 8.0 query:

```
WITH user_merchant AS (
  SELECT merchant_id,
         user_id,
         SUM(amount) AS user_volume
  FROM transactions
  GROUP BY merchant_id, user_id
), ranked_users AS (
  SELECT merchant_id,
         user_id,
         user_volume,
         ROW_NUMBER() OVER (
           PARTITION BY merchant_id
           ORDER BY user_volume DESC
         ) AS rn
  FROM user_merchant
), top_five AS (
  SELECT merchant_id,
         SUM(user_volume) AS top5_volume
  FROM ranked_users
  WHERE rn <= 5
  GROUP BY merchant_id
), merchant_totals AS (
  SELECT merchant_id,
         SUM(amount) AS total_volume
  FROM transactions
  GROUP BY merchant_id
)
SELECT t.merchant_id,
       t.top5_volume,
       m.total_volume,
       ROUND(t.top5_volume / m.total_volume, 4) AS top5_ratio
FROM top_five AS t
JOIN merchant_totals AS m
  ON m.merchant_id = t.merchant_id
WHERE t.top5_volume / m.total_volume > 0.60
ORDER BY top5_ratio DESC;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Pattern 9 · JUST-UNDER-THRESHOLD (STRUCTURING)

Users with 10+ transactions at exactly ■9,999.

Validated findings: 20 suspect result(s).

Top results: 14690 | 25; 14680 | 25; 14693 | 22

MySQL 8.0 query:

```
SELECT user_id,
       COUNT(*) AS threshold_txn_count
FROM transactions
WHERE amount = 9999.00
GROUP BY user_id
HAVING COUNT(*) >= 10
ORDER BY threshold_txn_count DESC;
```

Pattern 10 · DORMANT-THEN-ACTIVE

Users with a 90+ day gap between consecutive transactions and at least 15 transactions after the gap.

Validated findings: 26 suspect result(s).

Top results: 14526 | 55; 14708 | 28; 14701 | 28

MySQL 8.0 query:

```
WITH transaction_gaps AS (
  SELECT user_id,
         txn_time,
         LAG(txn_time) OVER (
           PARTITION BY user_id
           ORDER BY txn_time
         ) AS prev_txn_time
  FROM transactions
), dormant_gaps AS (
  SELECT user_id,
         txn_time AS reactivation_time,
         prev_txn_time
  FROM transaction_gaps
  WHERE prev_txn_time IS NOT NULL
        AND TIMESTAMPDIFF(DAY, prev_txn_time, txn_time) >= 90
), post_gap_activity AS (
  SELECT d.user_id,
         d.reactivation_time,
         COUNT(t.txn_id) AS post_gap_txns
  FROM dormant_gaps AS d
  JOIN transactions AS t
    ON t.user_id = d.user_id
   AND t.txn_time >= d.reactivation_time
  GROUP BY d.user_id, d.reactivation_time
  HAVING COUNT(t.txn_id) >= 15
)
SELECT user_id,
       MAX(post_gap_txns) AS post_gap_txns
FROM post_gap_activity
GROUP BY user_id
ORDER BY post_gap_txns DESC;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Pattern 11 · VELOCITY SPIKE

Users whose peak monthly transaction count is at least 5× their six-month average, with a peak of at least 20 transactions. The first active month is excluded from the anomaly comparison so a new user's first burst is not treated as a historical spike.

Validated findings: 42 suspect result(s).

Top results: 14511 | 2024-06 | 8.0 | 46 | 5.75; 14526 | 2024-05 | 9.333333333333334 | 53 | 5.678571428571428; 14527 | 2024-02 | 9.333333333333334 | 53 | 5.678571428571428

MySQL 8.0 query:

```
WITH monthly_counts AS (
    SELECT user_id,
           DATE_FORMAT(txn_time, '%Y-%m') AS txn_month,
           COUNT(*) AS monthly_txns
    FROM transactions
    GROUP BY user_id, DATE_FORMAT(txn_time, '%Y-%m')
), user_stats AS (
    SELECT user_id,
           SUM(monthly_txns) / 6.0 AS avg_monthly_txns,
           MAX(monthly_txns) AS peak_monthly_txns
    FROM monthly_counts
    GROUP BY user_id
), peak_months AS (
    SELECT m.user_id,
           m.txn_month,
           m.monthly_txns,
           s.avg_monthly_txns,
           s.peak_monthly_txns
    FROM monthly_counts AS m
    JOIN user_stats AS s
      ON s.user_id = m.user_id
     AND s.peak_monthly_txns = m.monthly_txns
), first_active_month AS (
    SELECT user_id,
           MIN(txn_month) AS first_month
    FROM monthly_counts
    GROUP BY user_id
)
SELECT p.user_id,
       p.txn_month AS peak_month,
       ROUND(p.avg_monthly_txns, 2) AS avg_monthly_txns,
       p.peak_monthly_txns,
       ROUND(p.peak_monthly_txns / p.avg_monthly_txns, 2) AS spike_ratio
FROM peak_months AS p
JOIN first_active_month AS f
  ON f.user_id = p.user_id
WHERE p.peak_monthly_txns >= 20
     AND p.peak_monthly_txns / p.avg_monthly_txns >= 5
     AND p.txn_month > f.first_month
ORDER BY spike_ratio DESC;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Pattern 12 · GEOGRAPHIC IMPOSSIBILITY

Users with consecutive transactions in different cities within 60 minutes.

Validated findings: 15 suspect result(s).

Top results: 14741; 14742; 14743

MySQL 8.0 query:

```
WITH ordered_transactions AS (
    SELECT user_id,
           city,
           txn_time,
           LAG(city) OVER (
               PARTITION BY user_id
               ORDER BY txn_time
           ) AS prev_city,
           LAG(txn_time) OVER (
               PARTITION BY user_id
               ORDER BY txn_time
           ) AS prev_txn_time
    FROM transactions
)
SELECT DISTINCT user_id
```

```
FROM ordered_transactions
WHERE prev_txn_time IS NOT NULL
      AND city <> prev_city
      AND TIMESTAMPTDIFF(MINUTE, prev_txn_time, txn_time) <= 60
ORDER BY user_id;
```

Why it works: This version uses the advanced technique requested by the project brief (self-join/EXISTS/CTE/window functions) rather than the simplified Week 3 alternative.

Submission Notes

1. Save the SQL submission as **RedFlag_Rishitha.sql**.
2. Run the dataset import first, then execute the solution file in MySQL Workbench.
3. The submission file intentionally excludes the CREATE TABLE and INSERT statements from the 18 MB dataset, as required by the brief.
4. Pattern 11 uses a six-month average and excludes a user's first active month from the spike comparison. This produces 42 flagged users on the supplied dataset, which falls inside the brief's expected 35–45 range.
5. Pattern 4 counts FAILED→SUCCESS pairs where the amount matches and the successful attempt occurs within two minutes. Pattern 6 requires CREDIT via NETBANKING followed by a UPI DEBIT within 30 minutes for at least 70% of the credit amount.

End of solution.